

Python for Technical Interviews

by Lee Zong Xun

Who am I?

I just finished my last semester, studying computer science and statistics.

- CVWO, Jupyterlab, TikTok, QRT.
- ex-VP of NUS DG, ex-Chair of SoC TIPS.
- Full time offers at Meta, QRT, TikTok etc.
- Currently in Hong Kong (Python Dev) 😊

Github: Zxun2 \ LinkedIn: Lee Zong Xun



Outline for Tonight

- Why Python?
- Crash Course to Python (for absolute beginners)
- Advanced Python (including useful libraries)
- Useful Techniques
- Python 3.9, 3.10, 3.11, 3.12 Features
- Performance

Why Python?

Generally, **language doesn't matter.**

You can use whatever language you're most comfortable with, so long your interviewer knows it (or at least can read and comprehend it), and you can solve all problems with it.

BUT I do believe that Python is a good choice of language for coding interviews, and comes with clear advantages.

- **Easier to code:** Spend your time on things that matter more.
- **Easier to read:** Make your interviewer's life easier.

But I'm new to Python. Will this session be helpful?

One important thing you'll realise along the way is - your understanding of how code works is **highly transferable!**

Every language has its quirks, but once you get past those, you'll see that all languages ultimately do the same few things.

Let's begin! 🌟🌟😊

Introduction to Python

Python is both a strongly typed and dynamically typed language.

Strongly typed: Variables have a type and the type matters when performing operations on a variable.

```
x = 3 + 10.5 # Valid, x = 13.5  
x = 1 + '4'  # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Dynamically typed: Type of the variable is only determined during runtime.

```
str_then_int = "Hello"  
str_then_int = 10      # No issue
```

Basic (Primitive) Types

- Numeric, Sequence, Text, Boolean, Binary, Mapping, Set, None
 - int (integer)
 - float (floating point number)
 - str (string): text
 - bool (boolean): yes / no
 - none: missing / unknown value

Numerics (1)

```
1 # Types: int, float
2 num_1 = 10
3 type(num_1)    # <class 'int'>
4
5 num_2 = 10.5
6 type(num_2)    # <class 'float'>
7
8 num_3 = num_1 + num_2
9 print(num_3)   # 20.5
10 type(num_3)   # <class 'float'>
11
12 # Other types: complex (not really useful)
13 three_five_i = complex(3, 5)
```

Numerics (2)

```
1 # Arithmetic operations
2 1 + 2          # 3
3 3 - 20         # -17
4 4 * 2          # 8
5 20 / 7         # 2.857142857142857
6 20 // 7        # 2
7 20 % 7         # 6
8 5 ** 3         # 125
9
10 # Comparators
11 10 <= 5        # False
12 5 > 2          # True
13 3 == 6        # False
14 10 != 5       # True
```

Booleans

```
1 # Values: True, False
2 x1 = True or False      # True
3 x2 = True and False     # False
4 x3 = not True           # False
5 x4 = not (True and False) # True
6 type(x1)                # <class 'bool'>
7
8 # bools are implemented using ints
9 True == 1               # True
10 False == 0             # True
```

Strings (1)

```
1 string_1 = "This is a string"
2 string_2 = 'This is also a string'
3
4 # String concatenation - O(total length)
5 string_1 + string_2      # 'This is a stringThis is also a string'
6 string_1 * 2             # 'This is a stringThis is a string'
7 'ba' + 'na' * 2         # 'banana'
8
9 # String indexing: [index] - O(1)
10 'Hello World'[0]        # 'H'
11 'Hello World'[3]        # 'l'
12 'Hello World'[-1]       # 'd'
```

No difference between strings and characters!

Strings (2)

```
1 # String slicing: [start:stop:step] - O(length of new string)
2 'Hello World'[0:3:1] # 'Hel'
3 'Hello World'[:3]    # 'Hel'
4 'Hello World'[:100]  # 'Hello World'
5 'Hello World'[3:]    # 'lo World'
6 'Hello World'[3:-4]  # 'lo W'
7 'Hello World'[:7:2]  # 'HloW'
8 'Hello World'[0::3]  # 'HlWl'
9 'Hello World'[::-1]  # 'dlroW olleH'
10
11 # in operator - O(n) on average, O(nm) worst case
12 'Hello' in 'Hello World' # True
13 'hello' in 'Hello World' # False
```

If you're keen on viewing the code for the in operator: [fastsearch.h](https://github.com/ericniebler/attrs/blob/master/src/attrs/setfuncs.py)

Strings (3)

```
1 # Comparison: lexicographic
2 'apple' < 'banana'      # True
3 'aaa' < 'aa'           # False
4
5 # Casing
6 'Hello World'.lower()   # 'hello world'
7 'hello world'.upper()   # 'HELLO WORLD'
8 'hello world'.title()   # 'Hello World'
9 'hello'.islower()       # True
10 'HI'.isupper()         # True
11 'Hello World'.istitle() # True
```

Strings (4)

```
1 # Getting length
2 len('Hello') # 5
3
4 # String splitting
5 'Hello+World+Bye'.split('+') # ['Hello', 'World', 'Bye']
6
7 # Digits / Alphabets
8 'Hello'.isalpha() # True
9 'H3llo'.isalpha() # False
10 '12345'.isnumeric() # True
11 'a2345'.isnumeric() # False
12 'a234s'.isalnum() # True
```

None

```
1 this_is_none = None
2
3 # Checking with identity comparison.
4 # None is always the same object.
5 this_is_none is None # True
```


Type Conversion

```
1 six_string = str(6)          # '6'
2 six_int = int('6')           # 6
3
4 another_six = int(6.3)        # 6
5 six_again = int(6.7)          # 6
6
7 six_float = float(6)           # 6.0
8 float_two = float('6.3')      # 6.3
9
10 int('Hello World')          # ValueError: invalid literal for int() with base 10:
11                             # 'Hello World'
```

Lists (1)

```
1 this_is_a_list = []
2 this_is_also_a_list = list()
3
4 # Append - O(1)
5 this_is_a_list.append(1) # [1]
6 this_is_a_list.append("Hello") # [1, 'Hello']
7
8 # Extend - O(length of second list)
9 this_is_a_list.extend([None, True, 10]) # [1, 'Hello', None, True, 10]
10
11 # List concatenation, new list created - O(total length)
12 this_is_a_list + [1, 2, 3] # [1, 'Hello', None, True, 10, 1, 2, 3]
```

Lists are mutable sequences!

Lists (2)

```
1 # List indexing: [index] - O(1)
2 this_is_a_list[0]      # 1
3 this_is_a_list[3]      # True
4 this_is_a_list[-1]     # 10
5
6 # Assignment
7 this_is_a_list[2] = 200
8 this_is_a_list        # [1, 'Hello', 200, True, 10]
9
10 # in operator - O(n)
11 'Hello' in this_is_a_list # True
12 'bye' in this_is_a_list  # False
```

Lists (3)

```
1 # List slicing: [start:stop:step] - O(length of new list)
2 this_is_a_list[0:3:1] # [1, 'Hello', 200]
3 this_is_a_list[:3]    # [1, 'Hello', 200]
4 this_is_a_list[3:]    # [True, 10]
5 this_is_a_list[3:-4]  # []
6 this_is_a_list[:5:2]  # [1, 200, 10]
7 this_is_a_list[0::3]  # [1, True]
8 this_is_a_list[::-1]  # [10, True, 200, 'Hello', 1]
9
10 # sort - O(n log n) - mutates original list
11 new_list = [10, 3, 5, 2, 20]
12 new_list.sort()
13 new_list # [2, 3, 5, 10, 20]
14 new_list.sort(reverse=True)
15 new_list # [20, 10, 5, 3, 2]
```

Lists (4)

```
1 # sorted - O(n log n) - returns new list
2 sorted(new_list) # [2, 3, 5, 10, 20]
3
4 # Length of list
5 len(new_list) # 5
6
7 # Pop from index and returns - O(n) worst case
8 # Mutates original list
9 new_list = [1, 2, 3, 4, 5]
10 new_list.pop(2) # 3
11 new_list # [1, 2, 4, 5]
12
13 # Reverse - O(n) - mutates original list
14 new_list.reverse()
15 new_list # [5, 4, 2, 1]
```

Lists (5)

```
1 # Sum of list - O(n)
2 sum(new_list) # 12
3
4 # Remove first match - O(n) worst case.
5 # Mutates original list
6 new_list = ['Hello', 'World', 'Goodbye', 'Hello']
7 new_list.remove('Hello')
8 new_list # ['World', 'Goodbye', 'Hello']
9 new_list.remove('Bye') # ValueError: list.remove(x): x not in list
10
11 # Insert - O(n) - mutates original list
12 index = 1
13 new_list.insert(index, 'Inserted')
14 new_list # ['Hello', 'Inserted', 'Goodbye', 'World']
```

Tuples

```
1 this_is_a_tuple = ()
2 this_is_also_a_tuple = tuple()
3
4 single_ele_tuple = (1,) # Cannot do (1)!
5 multi_ele_tuple = (1, 2, 3, 4, 5)
6
7 # Immutable
8 multi_ele_tuple[2] = 5 # TypeError: 'tuple' object does not support item
9                        # assignment
10
11 # Can be indexed, sliced and concatenated just like lists.
12 # All operations will produce a new tuple.
```

Tuples are immutable sequences, which allow them to be hashed and used as hash table keys or hash set elements!

Dictionaries (1)

```
1 dictionary = {}
2 another_one = dict()
3
4 # Insertion - O(1)
5 dictionary['key'] = 10 # {'key': 10}
6 dictionary[10] = 'value' # {'key': 10, 10: 'value'}
7
8 # Update - O(1)
9 dictionary['key'] = 20 # {'key': 20, 10: 'value'}
10
11 # Can insert with immutable keys, e.g. tuples
12 dictionary[('tuple',)] = 'is ok' # {'key': 20, 10: 'value', ('tuple',): 'is
13                                # ok'}
```

Python's hash tables!

Dictionaries (2)

```
1 # Getting value for a key - O(1)
2 dictionary[10] # 'value'
3 dictionary[9]  # KeyError
4
5 # Containment (key) check - O(1)
6 10 in dictionary # True
7 9 in dictionary  # False
8
9 # Other methods
10 dictionary.keys()    # dict_keys(['key', 10, ('tuple',)])
11 dictionary.values()  # dict_values([20, 'value', 'is ok'])
12 dictionary.items()   # dict_items([('key', 20), (10, 'value'), (('tuple',),
13                        # 'is ok')])
```

Functions

```
1 def function_name(param1, param2):  
2     return 10  
3  
4 def recursive(x):  
5     if x < 0:  
6         return 1  
7     return recursive(x - 1)  
8  
9 def func():  
10     def inner_func():  
11         return 10  
12     return inner_func  
13  
14 def hof(another_func, param):  
15     return another_func(param)
```

Control Structures

```
1 # Looping from 0 to 9
2 for i in range(10):
3     print(i) # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
4
5 # Looping from 5 to 10
6 for i in range(5, 11):
7     print(i) # 5, 6, 7, 8, 9, 10
8
9 # Looping from 10 to 1
10 # Same as slicing: range(start, stop, step)
11 for i in range(10, 0, -1):
12     print(i) # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

range returns to us a *Range* object! Similarly, can loop through *lists*, *tuples* and *dictionaries*. Anything that is an *iterable* / *iterator*.

Range Object

Implements the [collections.abc.Sequence](#) ABC, and provide features such as containment tests, element index lookup, slicing and support for negative indices.

```
>>> r = range(0, 20, 2)
>>> r
range(0, 20, 2)
>>> 11 in r
False
>>> 10 in r
True
>>> r[5]
10
>>> r[:5]
range(0, 10, 2)
```

Always take the *same* amount of memory, no matter the size of the range.

Iterator vs Iterable

An *iterable* is capable of returning its members one at a time, allowing it to be looped over in a for-loop. An iterable must implement the `_iter_()` method, which returns an iterator.

An *iterator* represents a stream of data. It returns the data one element at a time and keeps track of its current position. An iterator must implement two methods: `_iter_()` and `_next_()`. The `_next_()` method returns the next item in the sequence and raises a `StopIteration` exception when no more items are available.

Control Structures (2)

```
1 # While loops
2 while condition:
3     do_work()
4
5 while condition:
6     if another_condition:
7         continue
8     do_work()
9     if terminating_condition:
10         break
```

Control Structures (3)

```
1 # if-else. No switch statements in Python (before 3.10)!
2 if condition1:
3     print("Condition 1 is True")
4 elif condition2:
5     print("Condition 2 is True")
6 else:
7     print("Neither are true")
8
9 # One-liners (not recommended)
10 if condition1: print("This also works")
```

Truthy and Falsy Values

```
1 if not []:  
2     print('Empty list is falsy') # 'Empty list is falsy'  
3  
4 if not ():  
5     print('Empty tuple is falsy') # 'Empty tuple is falsy'  
6  
7 if not {}:  
8     print('Empty dict is falsy') # 'Empty dict is falsy'  
9  
10 if not '':  
11     print('Empty string is falsy') # 'String is empty'  
12  
13 if 20:  
14     print('Non-0 numbers are truthy') # 'Non-0 numbers are truthy'
```


Lambda Functions

```
1 lambda x: x + 10
2 lambda x, y: x + y
3
4 # Currying
5 add = lambda x: lambda y: x + y
6 add_3 = add(3)
7 add_3(5) # 8
```

```
Trie = lambda x: defaultdict(Trie) # Trie in a single line :o
```

Map & Filters

```
1 list(map(lambda x: x + 10, [1, 2, 3])) # [11, 12, 13]
2
3 list(filter(lambda x: x >= 10, [11, 9, 10, 8])) # [11, 10]
4
5 # For strings, you cannot cast using str().
6 # Instead, you need to .join() it.
7 ''.join(filter(lambda x: x.islower(), 'HeLlO')) # 'el'
8
9 # You also cannot directly use len() on a filter/map object
10 # You will need to convert it to something else e.g. list
11 len(filter(lambda x: x > 10, [11, 9, 10, 8])) # TypeError: object of type
12                                             # 'filter' has no len()
13 len(list(filter(lambda x: x >= 10, [11, 9, 10, 8]))) # 2
14
15 # You can sum a filter/map object
16 sum(filter(lambda x: x > 2, [1, 2, 3, 4])) # 7
```

List Comprehension

```
1 original_list = [1, 12, 9, 30, 5]
2
3 # Mapping
4 [x + 10 for x in original_list] # [11, 22, 19, 40, 15]
5 ['G' if x > 10 else 'B' for x in original_list] # ['B', 'G', 'B', 'G', 'B']
6
7 # Filtering
8 [x for x in original_list if x > 10] # [12, 30]
9
10 nested_list = [[1, 2], [3, 4, 5], [6]]
11
12 # List flattening
13 [ele for sublist in nested_list for ele in sublist] # [1, 2, 3, 4, 5, 6]
```

A nifty technique that makes processing of iterables very short and simple. However, it may compromise readability. It is potentially *faster*.

(Slightly) Advanced Python

Classes (1)

```
1 class Animal:
2     # Constructor
3     def __init__(self, name): # self is like Java's this
4         self.name = name
5
6 my_animal = Animal('Zong Xun')
7 print(my_animal.name)      # 'Zong Xun'
8
9 # You can assign properties during runtime
10 print(my_animal.height) # AttributeError: 'Animal' object has no attribute
11                        # 'height'
12 my_animal.height = 1.9
13 print(my_animal.height) # 1.9
```

Classes (2) - Inheritance

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5 # Subclassing
6 class Cat(Animal):
7     # Subclass Constructor
8     def __init__(self, name, weight):
9         super().__init__(name) # Call to superclass constructor
10        self.weight = weight
11
12 another_cat = Cat('Kittteh', 1.5)
13 print(another_cat.name) # 'Kittteh'
14 print(another_cat.weight) # 1.5
```

Classes (3) - Methods

```
1 class Cat(Animal):
2     def __init__(self, name, weight):
3         super().__init__(name)
4         self.weight = weight
5
6     def scratch(self):
7         print(f"{self.name} does a scratch!")
8
9 new_cat = Cat('Kittteh', 1.5)
10 new_cat.scratch()    # 'Kittteh does a scratch!'
```

Classes (4) - Private Methods

```
1 class Cat(Animal):
2     def __init__(self, name, weight):
3         super().__init__(name)
4         self.__weight = weight # class private
5
6     def _scratch(self):
7         print(f"{self.name} does a scratch!")
8
9 new_cat = Cat('Kittteh', 1.5)
10 new_cat._scratch() # 'Kittteh does a scratch!'
11 print(new_cat._Cat__weight) # 1.5
```


Classes (5) - Dunder Methods

```
1 class Cat(Animal):
2     def __str__(self): # Like Java's toString()
3         return f'Name: {self.name}, Weight: {self.weight}'
4
5     def __eq__(self, other):
6         if other.__class__ is self.__class__:
7             return (self.name, self.weight) == (other.name, other.weight)
8         else:
9             return NotImplemented
10    # In addition to __lt__, __le__, __ge__
11    def __gt__(self, other):
12        if other.__class__ is self.__class__:
13            return (self.name, self.weight) > (other.name, other.weight)
14        else:
15            return NotImplemented
```

View other methods, e.g. `len`, `add`, etc. [here!](#)

Common Classes (1)

```
1 class LinkedListNode:
2     def __init__(self, value, next = None):
3         self.value = value
4         self.next = next
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9         self.tail = None
10        self.count = 0
11
12    def append(self, new_value):
13        new_node = LinkedListNode(new_value)
14        self.count += 1
15        if self.tail is None: # and so on...
```

Common Classes (2)

```
1 # Binary Tree
2 class Node:
3     def __init__(self, value, left = None, right = None):
4         self.value = value
5         self.left = left
6         self.right = right
7
8 # General Graph
9 class Node:
10     def __init__(self, value):
11         self.value = value
12         self.neighbours = {} # OR a list []
13
14     def add_neighbour(self, neighbour, weight):
15         self.neighbours[neighbour] = weight
```

Sets (1)

```
1 set1 = {'apple', 'banana', 'cherry'}
2 set2 = set(('abc', 34, True, 40, 'male')) # Or set([x, y, ...])
3
4 # Containment check
5 'abc' in set2 # True
6
7 # Adding items
8 set1.add('orange')
9 set1 # {'cherry', 'orange', 'apple', 'banana'}
10 set1.update([10, 20])
11 set1 # {'cherry', 'orange', 'apple', 'banana', 10, 20}
12
13 # Unioning sets
14 set3 = set1.union(set2)
15 set3 # {True, 'banana', 34, 'apple', 'abc', 40, 'cherry', 'male', 'orange',
16      # 10, 20}
```

Sets (2)

```
1 # Remove items
2 set1.remove(10)    # Throws error if item does not exist
3 set1  # {'cherry', 'orange', 'apple', 'banana', 20}
4 set1.discard(33)   # No error thrown if item does not exist
5
6 # Intersection
7 set4 = {'orange', 'pineapple'}
8 set5 = set1.intersection(set4)
9 set5  # {'orange'}
10
11 # Difference
12 set4 = {'orange', 'pineapple'}
13 set6 = set4.difference(set1)
14 set6  # {'pineapple'}
```

Sets (3)

```
1 x.isdisjoint(y) # True if no items in x is also in y
2 x.issubset(y)   # True if all items in x are also in y
3 x.issuperset(y) # True if all items in y are also in x
4
5 x.symmetric_difference(y) # Set of items in x and y that are not in both
```

Frozen Sets

```
1 fset = frozenset([1, 2, 3, 1])
2 fset  # frozenset({1, 2, 3})
3
4 dictionary = {}
5 dictionary[{1, 2, 3}] = True  # TypeError: unhashable type: 'set'
6 dictionary[fset] = True
7 dictionary  # {frozenset({1, 2, 3}): True}
```

Immutable sets! Just like tuples, they can be hashed!

Standard Libraries for Data Structures

Concurrent Queue

```
1 from queue import Queue
2
3 my_queue = Queue()
4 my_queue.put('Item 1') # Equiv to my_queue.put('Item 1', block=True)
5 my_queue.put('Item 2')
6 my_queue.put('Item 3')
7
8 print(my_queue.qsize()) # 3
9 my_queue.get()          # 'Item 1', equiv to my_queue.get(block=True)
10 print(my_queue.qsize()) # 2
11
12 my_queue.put_nowait('Item 4')
13 my_queue.get_nowait()    # Item 4
```

Priority Queue

```
1 from queue import PriorityQueue
2
3 my_queue = PriorityQueue()
4 my_queue.put((2, 'Item 1'))
5 my_queue.put((3, 'Item 2'))
6 my_queue.put((1, 'Item 3'))
7
8 my_queue.get()           # (1, 'Item 3')
9 print(my_queue.qsize())  # 2
```

Deque

```
1 from collections import deque
2
3 my_deque = deque()
4 my_deque.append('Item 1') # appends to the right
5 my_deque.appendleft('Item 2')
6 my_deque.append('Item 3')
7
8 print(len(my_queue))      # 3
9 my_queue.pop()            # 'Item 3', pops from the right
10 my_queue.popleft()        # 'Item 2'
11 print(len(my_queue))     # 1
```

Implemented as a linked list.

Heapq

```
1 from heapq import heappush, heappop, heapify, nlargest, nsmallest
2
3 heap = [8, 7, 3, 50, 52, 49, 29, 37, 32]
4 heapify(heap) # Transforms in-place, O(n)
5
6 min_element = heap[0] # 3
7 heappush(heap, 25)
8 heap[0] # 3
9 heappop(heap) # 3
10 heap[0] # 7
11 heappush(heap, 5)
12 heap[0] # 5
13
14 nlargest(5, [8, 7, 3, 50, 52, 49, 29, 37, 32]) # [52, 50, 49, 37, 32]
15 nsmallest(3, [8, 7, 3, 50, 52, 49, 29, 37, 32]) # [3, 7, 8]
```

OrderedDict

```
1 from collections import OrderedDict
2
3 d = OrderedDict()
4 d['a'] = 1
5 d['b'] = 2
6 d['c'] = 3
7 ''.join(d.keys()) # 'abc'
8
9 d.move_to_end('b')
10 ''.join(d.keys()) # 'acb'
11
12 d.move_to_end('b', last=False)
13 ''.join(d.keys()) # 'bac'
14
15 d.popitem(last=False) # ('b', 2)
```

Useful for LRU cache implementation.

Example: LRU Cache

```
1 from collections import OrderedDict
2
3 class LRUCache:
4     def __init__(self, capacity: int):
5         self.cache = OrderedDict()
6         self.capacity = capacity
7     def get(self, key: int) -> int:
8         if key not in self.cache:
9             return -1
10        else:
11            self.cache.move_to_end(key)
12            return self.cache[key]
13    def put(self, key: int, value: int) -> None:
14        self.cache[key] = value
15        self.cache.move_to_end(key)
16        if len(self.cache) > self.capacity:
17            self.cache.popitem(last=False)
```

Counters

```
1 from collections import Counter
2
3 counter = Counter()
4
5 for char in 'Count some freq':
6     counter[char] += 1
7
8 counter # Counter({'o': 2, ' ': 2, 'e': 2, 'C': 1, 'u': 1, 'n': 1, 't': 1,
9         # 's': 1, 'm': 1, 'f': 1, 'r': 1, 'q': 1})
```

Best for frequency counting. Defaults to 0 for new keys.

Default Dict

```
1 from collections import defaultdict
2
3 d_int = defaultdict(int)
4 d_list = defaultdict(list)
5
6 d_func = defaultdict(lambda: 'Default Value')
7
8 d_int['new key'] # 0
9 d_list['new key'] # []
10 d_func['new key'] # 'Default Value'
```

Define your own default values for keys not in the dictionary!

Standard Math Lib / Operations

Math

```
1 import math
2
3 math.ceil(10.5)    # 11
4 math.floor(10.5)   # 10
5 math.sqrt(16)      # 4.0
6 math.log(10)       # 2.302585092994046
7 math.log10(10)     # 1.0
8 math.log2(16)      # 4.0
9 math.inf, math.e, math.pi
10
11 # Not under the math library - works with filter/map objects!
12 min([1, 2, 3, 4, 5]) # 1
13 max([1, 2, 3, 4, 5]) # 5
14 sum([1, 2, 3, 4, 5]) # 15
```

Working with Representations

ASCII / CTOI / ITOC

```
1 a = chr(65)
2 print(a) # 'A'
3
4 sixty_five = ord('A')
5 print(sixty_five) # 65
6
7 ord('Hi') # TypeError: ord() expected a character, but string of length 2
8          # found
```

Different Bases

```
1 # Binary string to decimal int
2 decimal = int('10101', 2)
3 decimal # 21
4
5 # Decimal int to binary string
6 binary = bin(21)
7 binary # '0b10101' - can do binary.replace('0b', '') to remove prefix
8
9 # Decimal int to octal string
10 octal = oct(21)
11 octal # '0o25' - can do octal.replace('0o', '') to remove prefix
```

Typing Library

Typing (1)

```
1 from typing import List, Tuple, Dict, Union
2
3 x: int = 10
4 s: str = 'Hello World'
5
6 def add_two_numbers(x: int, y: int) -> int:
7     return x + y
8
9 def sum_iter(iter: Union[List[int], Tuple[int]]) -> int:
10     return sum(iter)
11
12 freq_count: Dict[str, int] = {'a': 1, 'b': 2, 'c': 3}
```

These are type hints. Not enforced during runtime, mostly for developer experience.

Typing (2)

```
1 from typing import Iterable, Dict, Union
2
3 def sum_iter(iter: Iterable[int]) -> int:
4     return sum(iter)
```


Typing (3)

```
1 from typing import Dict, Optional
2
3 class Node: # Binary Tree
4     # You cannot directly use Node as a type as it hasn't been evaluated yet!
5     def __init__(self, value: int, left: Optional['Node'] = None,
6                 right: Optional['Node'] = None) -> 'Node':
7         self.value: int = value
8         self.left: Optional[Node] = left
9         self.right: Optional[Node] = right
10
11 class Node: # General Graph
12     def __init__(self, value: int) -> 'Node':
13         self.value: int = value
14         self.neighbours: Dict[int, Node] = {}
15
16     def add_neighbour(self, neighbour: 'Node', weight: int) -> None:
17         self.neighbours[neighbour] = weight
```

Typing (4)

```
1 from typing import List
2
3 def largest_number(numbers: List[int]) -> int:
4     return -1
5
6 def maximum_subarray(arr: List[int]) -> int:
7     return -1
```

Function signatures - easy, but interviewers love it.

Enums

```
1 from enum import Enum
2
3 class Color(Enum):
4     RED = 1
5     GREEN = 2
6     BLUE = 3
7
8 red = Color.RED
9 print(red.name)    # 'RED'
10 print(red.value)  # 1
11 Color(1)          # <Color.RED: 1>
12 Color['RED']       # <Color.RED: 1>
13
14 class Weekday(Enum):
15     MONDAY = 'monday'
```

Decorators (1)

```
1 def uppercase_decorator(function):
2     def wrapper():
3         func = function()
4         uppercase = func.upper()
5         return uppercase
6
7     return wrapper
8
9 @uppercase_decorator
10 def say_hi():
11     return 'hello there'
12
13 say_hi()  # 'HELLO THERE'
```

A decorator is a design pattern in Python that allows a user to add new functionality to an existing object without modifying its structure.

Decorators (2) - "Class" and "Static" methods

```
1 class A:
2     def foo(self, x):
3         print(f"foo({self}, {x})")
4
5     @classmethod
6     def class_foo(cls, x):
7         print(f"class_foo({cls}, {x})")
8
9     @staticmethod
10    def static_foo(x):
11        print(f"static_foo({x})")
```

Class methods implicitly receive the class as the first argument. Static methods are more for code organisation - you can just write normal functions.

Decorators (3) - Instant Memoization

```
1 def fib(n):
2     if n <= 1:
3         return n
4     return fib(n - 1) + fib(n - 2)
5
6 fib(40) # Takes around 30s or so
7
10 @cache
11 def fib2(n):
12     if n <= 1:
13         return n
14     return fib2(n - 1) + fib2(n - 2)
15
16 fib2(400) # Instant!
```

Useful for DP problems! Can also be used for Singleton pattern.

Decorators (4) - Limited Memory

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=3)
4 def fib3(n):
5     if n <= 1:
6         return n
7     return fib3(n - 1) + fib3(n - 2)
8
9 fib3(400) # Also instant
```

Useful Techniques

String Building - Slow

```
1 # O(n^2) string building
2 chars = ['h', 'e', 'l', 'l', 'o']
3 result = ''
4 for char in chars:
5     result += char
6
7 result # 'hello'
```

String Building - Fast

```
1 # O(n) string building
2 ''.join(['h', 'e', 'l', 'l', 'o'])
3
4 # O(n) string reversal
5 string[::-1]
6 # OR
7 def reverse(string):
8     chars = []
9
10    # Loop from last character to first character
11    for i in range(len(string) - 1, -1, -1):
12        char = string[i]
13        chars.append(char)
14
15    return ''.join(chars)
```

Keep your code short (KYCS)

```
1 a, b = [1, 2]
2 a # 1
3 b # 2
4
5 a, b = [1, 2, 3] # ValueError: too many values to unpack (expected 2)
6
7 a, b = 1, 2 # 1, 2 implicitly creates a tuple (1, 2), which is then unpacked
8
9 a = b = 0
10 a += 1
11 a # 1
12 b # 0
13
14 # TAKE NOTE: The same instance is assigned to both!
15 a = b = []
16 a.append(10)
17 b # [10]
```

Enumerate

```
1 lst = ['a', 'b', 'c', 'd', 'e']
2
3 # Instead of:
4 for index in range(len(lst)):
5     ele = lst[index]
6     do_something(index, ele)
7
8 # You should do:
9 for index, ele in enumerate(lst):
10     do_something(index, ele)
```

If you need to loop through a sequence AND work with indices, use ***enumerate***.

Summary

You won't need **most** of the stuff all the time.

However, you may need some of them sometimes, and using these things demonstrate a strong proficiency in your language of preference.

No need to "memorise" them - just use them whenever you can during your regular LeetCode practices, and they will come to you naturally during the interviews.

Python 3.9

Dictionary Union

```
1 d = {'spam': 1, 'eggs': 2, 'cheese': 3}
2 e = {'cheese': 'cheddar', 'aardvark': 'Ethel'}
3
4 # In the past, you had to do:
5 {**d, **e} # {'spam': 1, 'eggs': 2, 'cheese': 'cheddar', 'aardvark': 'Ethel'}
6
7 # Now you can do:
8 d | e # {'spam': 1, 'eggs': 2, 'cheese': 'cheddar', 'aardvark': 'Ethel'}
9 e | d # {'cheese': 3, 'aardvark': 'Ethel', 'spam': 1, 'eggs': 2}
10
11 # Or in-place:
12 d |= e
13 d # {'spam': 1, 'eggs': 2, 'cheese': 'cheddar', 'aardvark': 'Ethel'}
```

No need to import List, Dict from Typing

```
1 from typing import Union
2
3 def sum_iter(iter: Union[list[int], tuple[int]]) -> int:
4     return sum(iter)
5
6 freq_count: dict[str, int] = {'a': 1, 'b': 2, 'c': 3}
```


Python 3.10

Structural Pattern Matching

```
1 match subject:  
2     case <pattern_1>:  
3         <action_1>  
4     case <pattern_2>:  
5         <action_2>  
6     case <pattern_3>:  
7         <action_3>  
8     case _: # wildcard  
9         <action_wildcard>
```

More complex than it seems! Not your usual switch-case statement!

Structural Pattern Matching (2)

```
1 # point is an (x, y) tuple
2 match point:
3     case (0, 0):
4         print("Origin")
5     case (0, y):
6         print(f"Y={y}")
7     case (x, 0):
8         print(f"X={x}")
9     case (x, y):
10        print(f"X={x}, Y={y}")
11    case _:
12        raise ValueError("Not a point")
```

Unpacking assignments can be done!

Structural Pattern Matching (3)

```
1 class Point:
2     x: int
3     y: int
4
5 # Works with **class** attributes
6 def location(point):
7     match point:
8         case Point(x=0, y=0):
9             print("Origin is the point's location.")
10        case Point(x=0, y=y):
11            print(f"Y={y} and the point is on the y-axis.")
12        case Point(x=x, y=0):
13            print(f"X={x} and the point is on the x-axis.")
14        case Point():
15            print("The point is located somewhere else on the plane.")
16        case _:
17            print("Not a point")
```

Structural Pattern Matching (4)

```
1 match points:
2     case []:
3         print("No points in the list.")
4     case [Point(0, 0)]:
5         print("The origin is the only point in the list.")
6     case [Point(x, y)]:
7         print(f"A single point {x}, {y} is in the list.")
8     case [Point(0, y1), Point(0, y2)]:
9         print(f"Two points on the Y axis at {y1}, {y2} are in the list.")
10    case _:
11        print("Something else is found in the list.")
```

Structural Pattern Matching (5)

```
1 match test_variable:
2     case ('warning', code, 40):
3         print("A warning has been received.")
4     case ('error', code, _):
5         print(f"An error {code} occurred.")
```

```
1 match point:
2     case Point(x, y) if x == y:
3         print(f"The point is located on the diagonal Y=X at {x}.")
4     case Point(x, y):
5         print(f"Point is not on the diagonal.")
```

No need to import Union, Optional etc

```
1 def sum_iter(iter: list[int] | tuple[int]) -> int:
2     return sum(iter)
3
4 freq_count: dict[str, int] = {'a': 1, 'b': 2, 'c': 3}
5
6 class Node: # Binary Tree
7     def __init__(self, value: int, left: 'Node' | None = None,
8                 right: 'Node' | None = None) -> 'Node':
9         self.value: int = value
10        self.left: Node | None = left
11        self.right: Node | None = right
```

Python 3.11

New built-in type - Self

```
1 from typing import Self
2
3 class Node: # General Graph
4     def __init__(self, value: int) -> Self:
5         self.value: int = value
6         self.neighbours: Dict[int, Node] = {}
7
8     def add_neighbour(self, neighbour: Self, weight: int) -> None:
9         self.neighbours[neighbour] = weight
```

Performance

Python 3.11 is between 10-60% faster than Python 3.10. On average, a 1.25x speedup on the standard benchmark suite. See [Faster CPython](#) for details.

Python 3.12

Generics (1)

More compact and explicit way to create generic classes and functions:

```
# Old
from typing import TypeVar, Generic

T = TypeVar('T')
class LoggedVar(Generic[T]):
    def __init__(self, value: T, name: str, logger: Logger) -> None:
        ...

# New
def max[T](args: Iterable[T]) -> T:
    ...

class list[T]:
    def __getitem__(self, index: int, /) -> T:
        ...
```

Generics (2)

More Examples

```
type IntFunc[**P] = Callable[P, int] # ParamSpec
type LabeledTuple[*Ts] = tuple[str, *Ts] # TypeVarTuple
type HashableSequence[T: Hashable] = Sequence[T] # TypeVar with bound
type IntOrStrSequence[T: (int, str)] = Sequence[T] # TypeVar with constraints
```

Performance Tips

CPython vs Cython

CPython and Cython are both related to Python, but they have distinct roles. CPython is the standard Python **interpreter**, while Cython is a language that allows writing Python-like code that can be compiled to C code, offering C-like performance. Essentially, CPython interprets and executes Python code, while Cython compiles Python code into C for faster execution.

Pytorch, Numpy and Pandas are libraries written in Cython.

Oversimplification

1. If you're willing to add a compilation step, you can get a 2-3x speedup on your existing python code.
2. If you're willing to do (1), and type the variables and functions used in your code, you can get a 10x speedup.
3. If you're willing to do (1) and (2), and spend some time thinking about your code and some computer science ideas, you can get a significant (50x or more) speedup.

Faster Python! 😊

Global Interpreter Lock

CPython 3.13 has experimental support for running with the [global interpreter lock](#) disabled. See [Free-threaded CPython](#) for more details.

Use with caution! Big performance cost on single-threaded performance.

Last Words

You only need one offer!

You have received an offer from **Meta** Inbox x



The Recruiting team at Meta  [<noreply@recruiting.facebook.com>](mailto:noreply@recruiting.facebook.com)
to me ▼

[Unsubscribe](#)